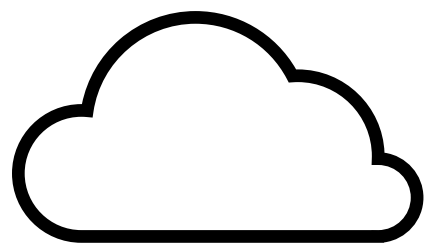


What are Small Language Models?

SLM

50M – 3B Params



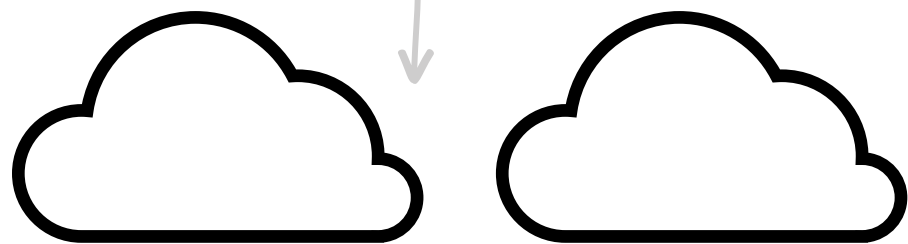
use small Cluster

Single CPU/GPU

\$0.05 – \$0.50

LLM

10B – 1T+ Params



use Large Cluster

Single CPU/GPU

\$5 – \$60

Cost / 1M Tokens

Small Language Models (SLMs) are **compact versions of Large Language Models (LLMs)**, typically with **< 10B Parameters**

Prompt

Write a social media post for
"SLMs vs LLMs"



SLM

2B Params



Given the same prompt, an SLM with around 2 billion parameters and an LLM with hundreds of billions of parameters **can produce comparable outputs.**

Prompt

Write a social media post for
"SLMs vs LLMs"



LLM

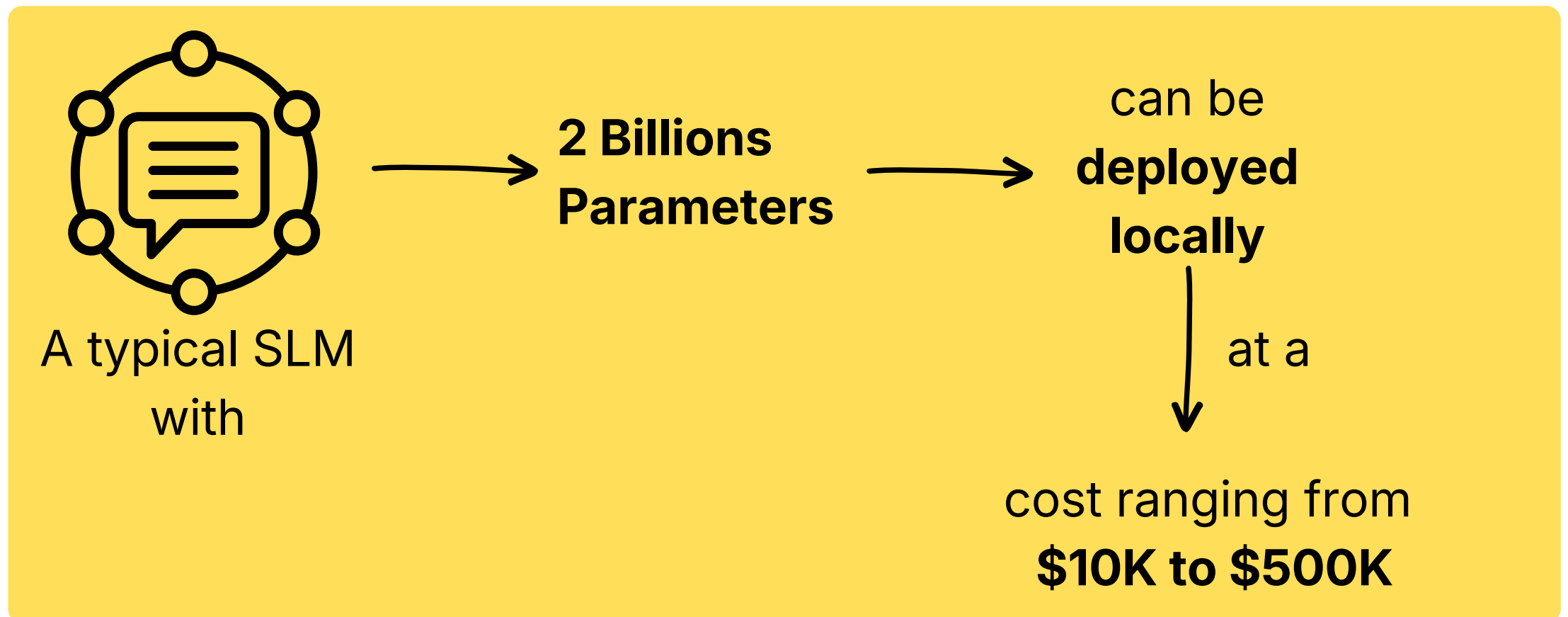
900B Params



Despite their much smaller size, SLMs can achieve similar performance when applied to focused, task-specific problems.



Small Language Models (SLMs) are especially practical for **real-world use**.



That's why SLMs stand out:

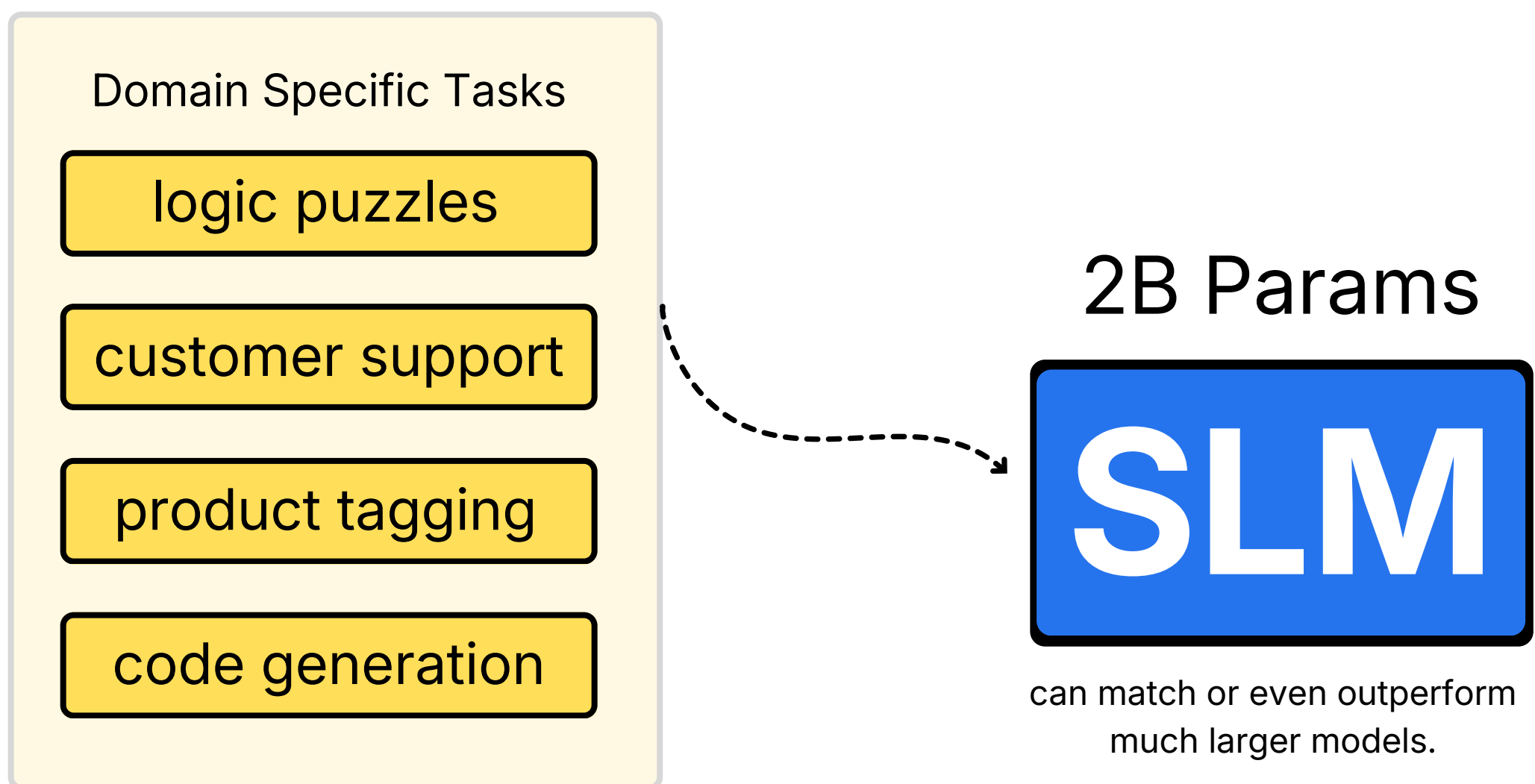
- They're easy to host
- Affordable
- Operationally lightweight.



SLMs as a **key driver of agentic AI** going forward, which raises an important question: if that's the case, where do large language models fall short?

Large Language Models excel at broad, general-purpose knowledge, which is why they require **hundreds of billions of parameters**

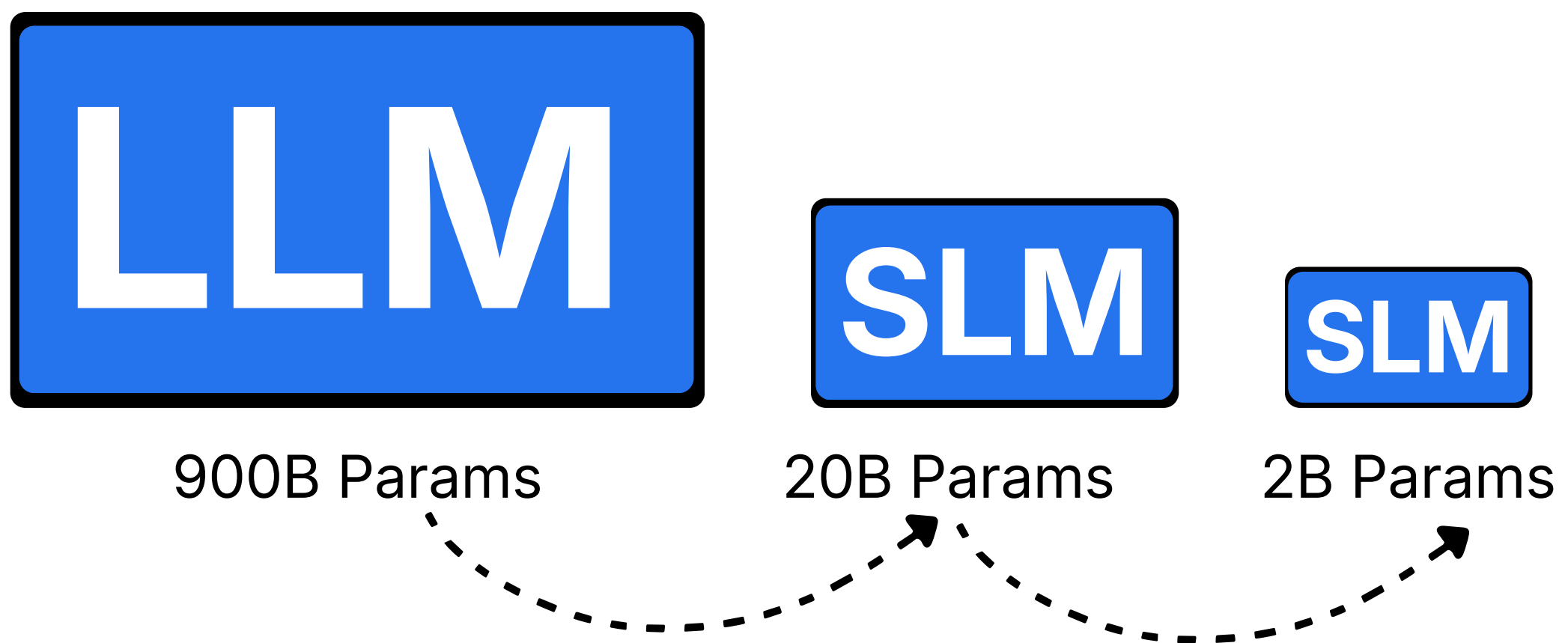
Small Language Models, on the other hand, shine when the problem is **well-defined**.



LLMs are built for breadth, while SLMs are built for focus. When specialization matters, smaller models often win.

How Did Language Models Get So Small?

SLMs didn't appear by magic. They are the result of several optimization techniques layered together.



These optimization techniques are listed below:

Quantization

Quantization reduces numerical precision of model weights.

LoRA Fine-Tuning

LoRA avoids retraining the entire model.

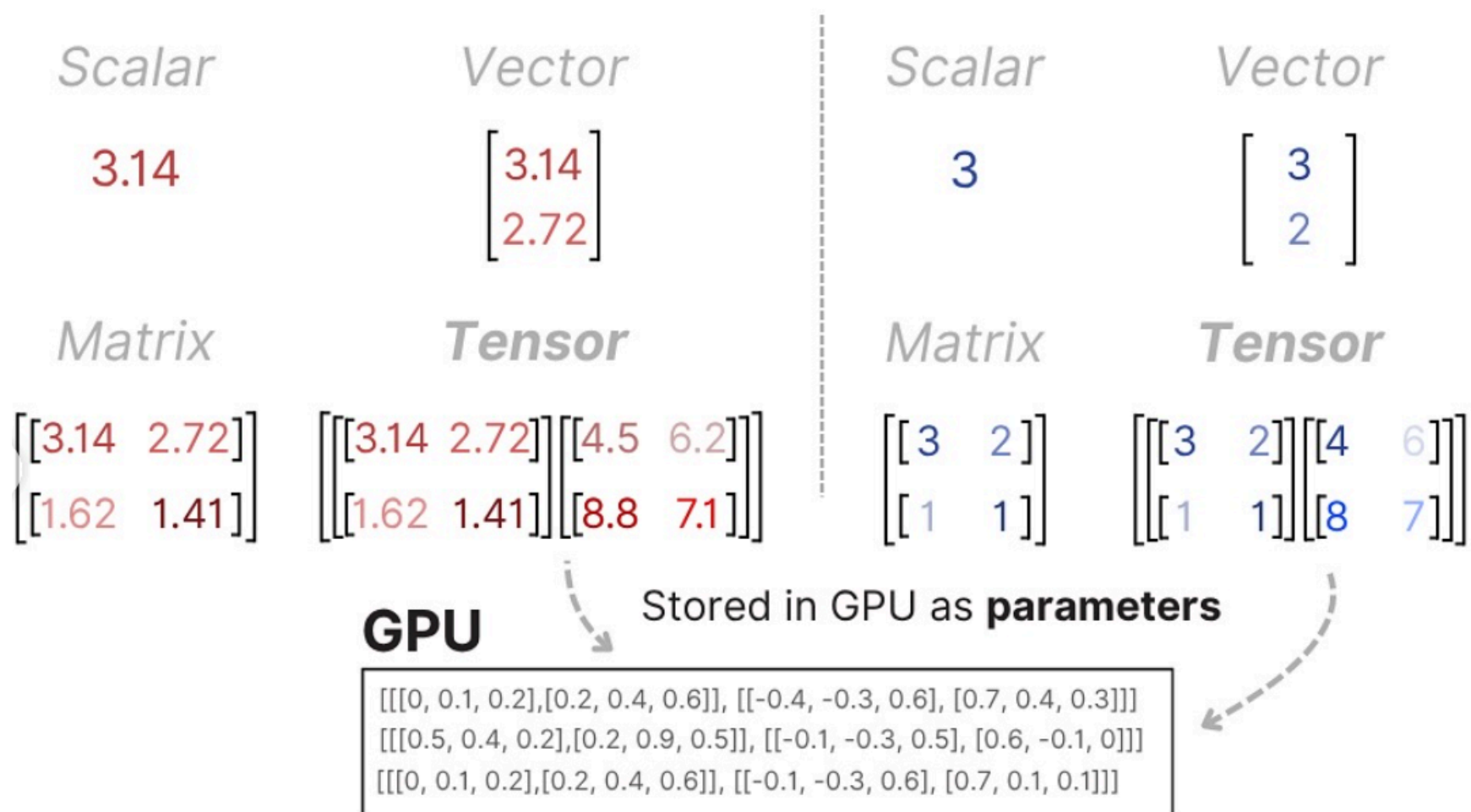
Flash Attention

Attention is expensive. Flash Attention rewrites how attention is computed.

Quantization

High-Precision (e.g., FP32)

Quantized (e.g., INT8)



Quantization reduces numerical precision of model weights.

Example

- FP32 → FP16 → INT8 → INT4

Impact

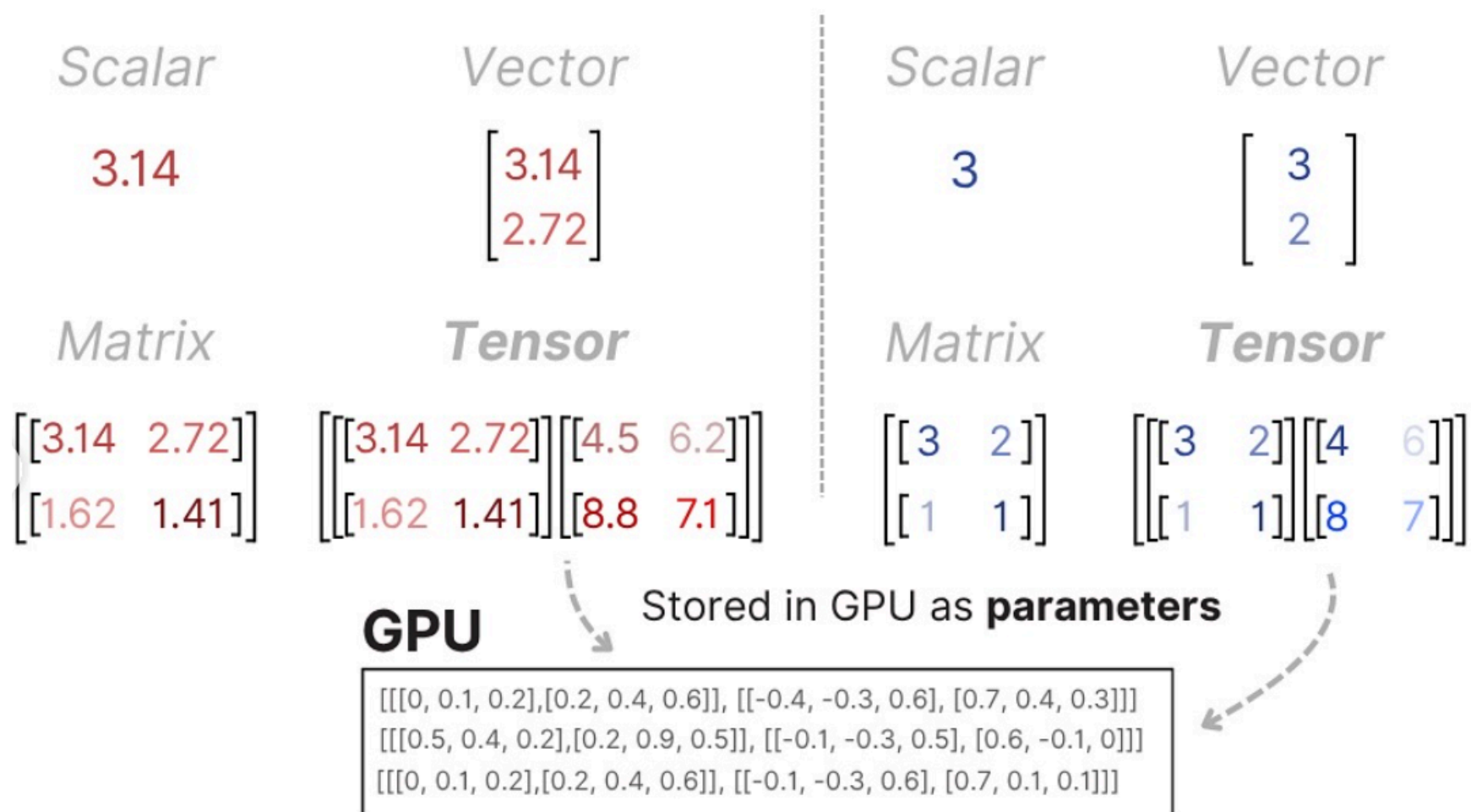
- 4x to 8x memory reduction
- Faster inference
- Minimal accuracy drop for many tasks

Modern quantization-aware training preserves output quality surprisingly well, especially for inference-only workloads.

Quantization

High-Precision (e.g., FP32)

Quantized (e.g., INT8)



Quantization reduces numerical precision of model weights.

Example

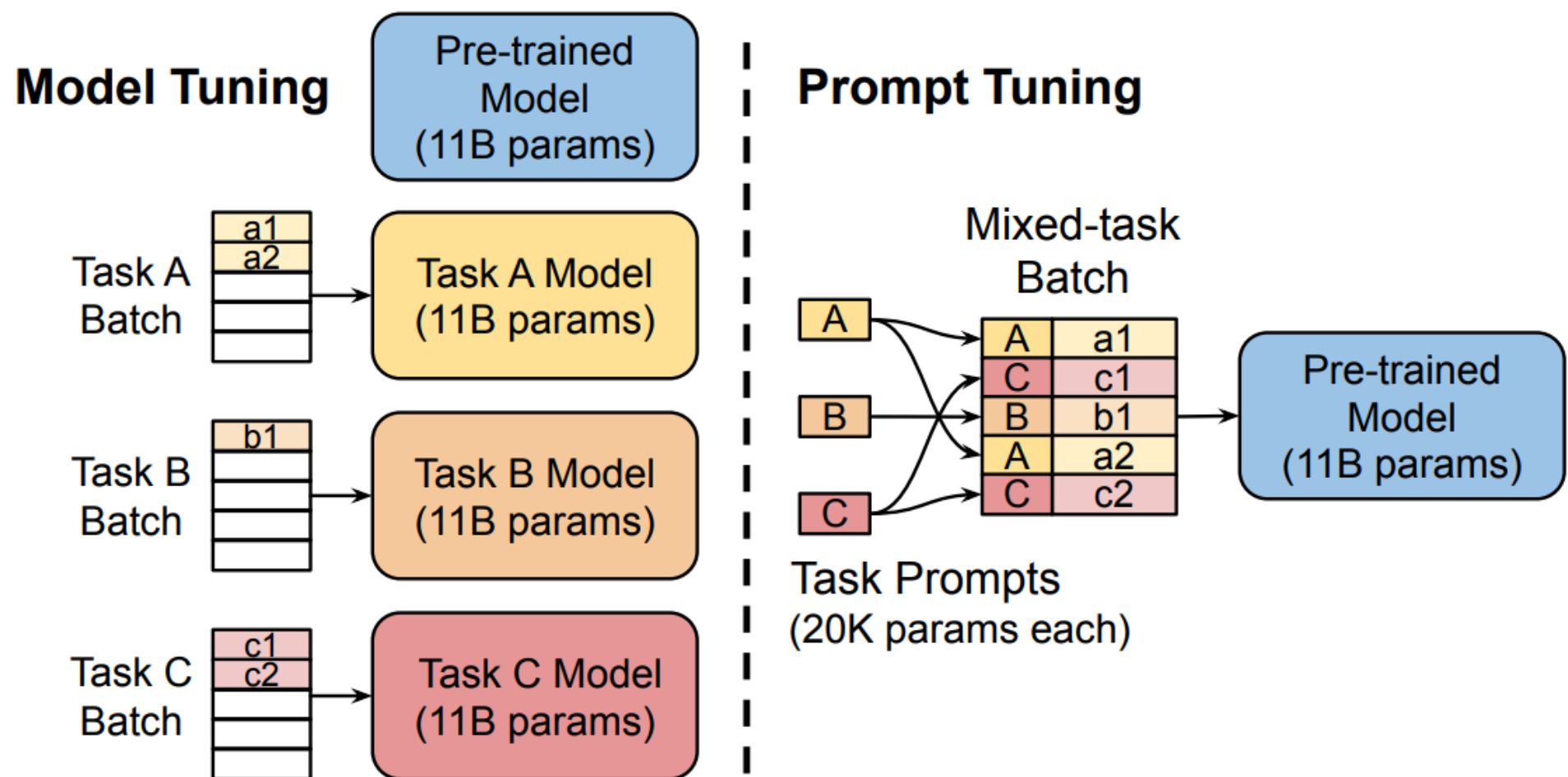
- FP32 → FP16 → INT8 → INT4

Impact

- 4x to 8x memory reduction
- Faster inference
- Minimal accuracy drop for many tasks

Modern quantization-aware training preserves output quality surprisingly well, especially for inference-only workloads.

Low-Rank Adaptation (LoRA) Fine-Tuning



LoRA avoids retraining the entire model.

Instead

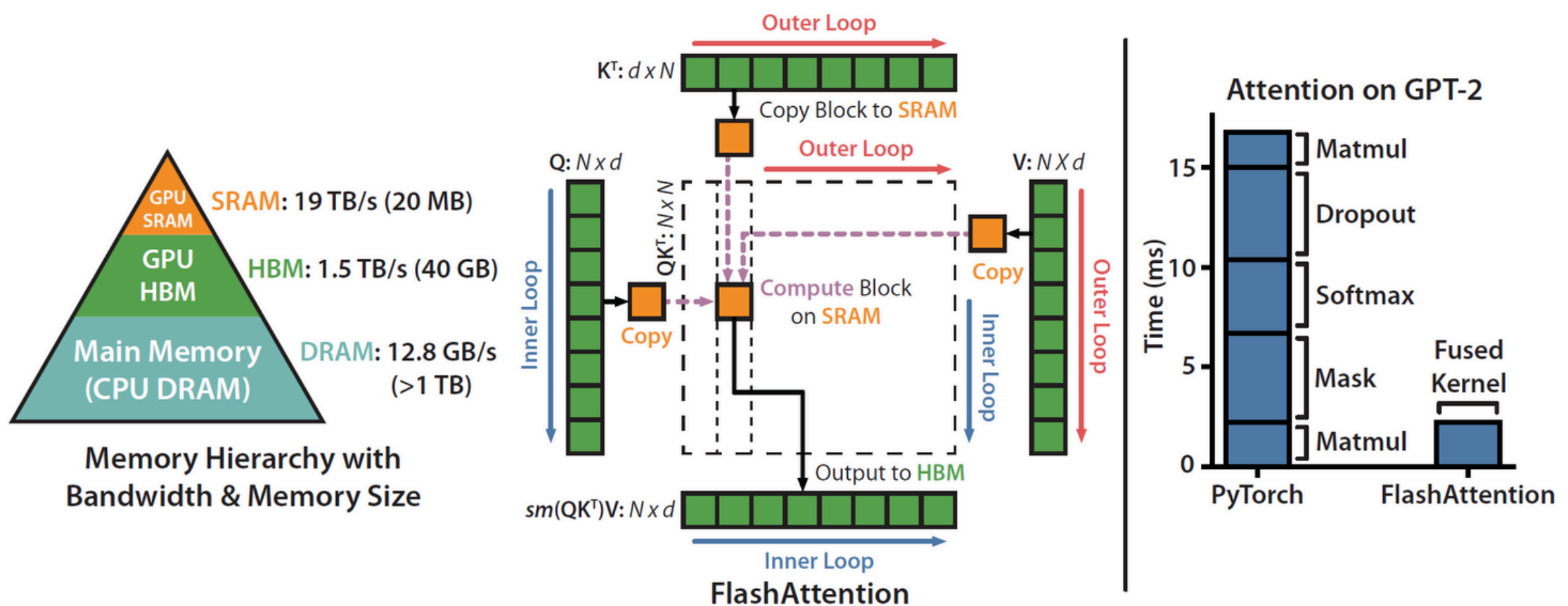
- Freeze base weights
- Train small low-rank matrices
- Inject task-specific behavior efficiently

Why this matters

- Fine-tuning costs drop drastically
- Model size stays small
- Multiple task adapters can share one base model

This is one of the biggest reasons SLMs are practical at scale.

Flash Attention



Attention is expensive. Flash Attention rewrites how attention is computed.

What it improves

- Memory efficiency
- Cache utilization
- Speed for long sequences

Why SLMs benefit

Even smaller models bottleneck on attention for long contexts. Flash Attention removes that bottleneck, letting SLMs handle longer inputs than their size would normally allow.